

FSCT8561-Lab3-AlvinSin

Part 2

```
def hash_password(password):  
  
    return hashlib.sha256(  
        password.encode()  
    ).hexdigest()  
  
def verify_password(password, stored_hash):  
  
    entered_hash = hash_password(  
        password  
    )  
  
    if entered_hash == stored_hash:  
        print("Password accepted")  
        return True  
    else:  
        print("Password rejected")  
        return False  
  
stored_hash = hash_password("Cyber123!")  
verify_password = verify_password("Cyber123!", stored_hash)  
(Python 3.7.4) C:\Users\alvin> cd C:\Users\alvin\OneDrive\Documents\FSCT8561-Lab3  
● Labs-Alvin-Sin/Lab3$ python3 auth_server.py  
Password accepted  
(Python 3.7.4) C:\Users\alvin>
```

Part 5

```
● Labs-Alvin-Sin/Lab3$ python3 auth_server.py  
TOTP secret:  
2ZGXYMX724LBC7NRH2XTCLT2YUZ5L2DI  
(Hard-Disk) costhm@costhm-IdeaPad-5-15IIL05:/media/usb1$ cat totp_secret.txt  
● Labs-Alvin-Sin/Lab3$ python3 auth_server.py  
TOTP secret:  
INKOZPABB5VUJ6WVUYW3UH426ZLE6UR3
```

No it does not generate the same secret

Part 6

```
import time

secret = pyotp.random_base32()

totp = pyotp.TOTP(secret)

current_otp = totp.now()

print("Current OTP:", current_otp)

def verify_otp(secret, otp):
    totp = pyotp.TOTP(secret)
    return totp.verify(otp)

# 1. Verify the current OTP
user_otp = input("Enter the current OTP: ")

if verify_otp(secret, user_otp):
    print("OTP accepted")
else:
    print("Invalid OTP")

# 2. Test an incorrect six-digit value
incorrect_otp = input("Enter an incorrect six-digit OTP: ")

if verify_otp(secret, incorrect_otp):
    print("OTP accepted")
else:
    print("Invalid OTP")

# 3. Test a previously generated OTP after it expires
old_otp = totp.now()

print("Wait for the OTP to expire...")
```

```

time.sleep(31)

if verify_otp(secret, old_otp):
    print("Old OTP accepted")
else:
    print("Old OTP expired or invalid")

```

● Labs-Alvin-Sin/Lab3\$ python3 auth_server.py
 Current OTP: 907723
 Enter the current OTP: 907723
 OTP accepted
 Enter an incorrect six-digit OTP: 5
 Invalid OTP
 Wait for the OTP to expire...
 Old OTP expired or invalid

Part 7

Combining hashing password and totp secret together, updating database logic

```

import hashlib, time, pyotp

# Part 2
def hash_password(password):

    return hashlib.sha256(
        password.encode()
    ).hexdigest()

def verify_password(password, stored_hash):

    entered_hash = hash_password(
        password
    )

    if entered_hash == stored_hash:
        print("Password accepted")
        return True
    else:
        print("Password rejected")
        return False

```

```

stored_hash = hash_password("Cyber123!")
verify_password = verify_password("Cyber123!", stored_hash)

# Generate TOTP Secret and user db
secret = pyotp.random_base32()

user = {
    "alice": {
        "password_hash": stored_hash,
        "totp_secret": secret
    }
}

print(
    "Alice's TOTP secret:",
    user["alice"]["totp_secret"]
)

# setting TOTP, TOTP verification
def verify_otp(secret, otp):
    totp = pyotp.TOTP(secret)
    return totp.verify(otp)

# Create OTP with Alice's secret
totp = pyotp.TOTP(user["alice"]["totp_secret"])
current_otp = totp.now()
print("current otp: ", current_otp)

# 1. Verify the current OTP
user_otp = input("Enter the current OTP: ")

if verify_otp(secret, user_otp):
    print("OTP accepted")
else:
    print("Invalid OTP")

# 2. Test an incorrect six-digit value
incorrect_otp = input("Enter an incorrect six-digit OTP: ")

```

```

if verify_otp(secret, incorrect_otp):
    print("OTP accepted")
else:
    print("Invalid OTP")

# 3. Test a previously generated OTP after it expires
old_otp = totp.now()

print("Wait for the OTP to expire...")
time.sleep(31)

if verify_otp(secret, old_otp):
    print("Old OTP accepted")
else:
    print("Old OTP expired or invalid")

```

```

(Hard-Disk) costnm@costnm-IdeaPad-5-1511L05:/media/costnm$ python3 auth_server.py
Password accepted
Alice's TOTP secret: AIGZCNG2EBU0JOE5HAINR5QJCWQEB62F
current otp: 986928
Enter the current OTP: 

```

Part 8

```

$ python3 auth_server.py
current otp: 568015
otpauth://totp/FSCT8561-Lab3:alice?secret=WQ2XTGRW2X6EJXYF4S3UREIKNSNL3X7L&issuer=FSCT8561-Lab3
Enter the current OTP: Invalid OTP

```

Part 9 to 11, part 13 tests

Test 1

```
$ python3 auth_client.py
Connected to Auth server
Username: alice
Password:
Username: alice
Password was collected without echoing it.
Server: OTP_REQUIRED
Enter OTP: 453402
Server: ACCESS_GRANTED
Authentication successful
You can now send message to server
Enter message or type EXIT to leave: hi there
Server: OK|Message received from alice
Enter message or type EXIT to leave: 
```

```
$ python3 auth_server.py
Server is waiting for a connection.
Connected by: ('127.0.0.1', 50182)
Received: AUTH|alice|Cyber123!
Current OTP: 453402
Password verified for: alice
Received: OTP|453402
OTP received from client: 453402
alice successfully authenticated
Received: MSG|hi there
alice says: hi there
```

Test 2

```
$ python3 auth_server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 50780)
Received: AUTH|alice|asd
Password verification failed for: alice
Client disconnected unexpectedly
Server closed
```

```

(hard-disk) C:\Users\user> python3 auth_client.py
Connected to Auth server
Username: alice
Password:
Username: alice
Password was collected without echoing it.
Server: ACCESS_DENIED
Auth failed
Connection closed.

```

Test 3

```

(hard-disk) C:\Users\user> python3 auth_client.py
Connected to Auth server
Username: alice
Password:
Username: alice
Password was collected without echoing it.
Server: OTP_REQUIRED
Enter OTP: k
Server: ERROR|OTP must be six digits
Auth failed
Connection closed.

```

```

(hard-disk) C:\Users\user> python3 auth_server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 53544)
Received: AUTH|alice|Cyber123!
Current OTP: 087360
Password verified for: alice
Received: OTP|k
OTP must be exactly 6 digits
Client disconnected unexpectedly
Server closed

```

Test 4

```

(hard-disk) C:\Users\user> python3 auth_server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 43118)
Received: AUTH|alice|Cyber123!
Current OTP: 335335
Password verified for: alice
Received: OTP|335335
OTP received from client: 335335
Invalid or expired OTP for: alice
Client disconnected unexpectedly
Server closed

```

```

(hard disk) COS tmm@cos tmm-idear-2019-10-11-05:/media/c
• $ python3 auth_client.py
Connected to Auth server
Username: alice
Password:
Username: alice
Password was collected without echoing it.
Server:  OTP REQUIRED
Enter OTP: 335335
Server:  ERROR ACCESS DENIED
Auth failed
Connection closed.

```

Test 5

```

• $ python3 auth_client.py
Connected to Auth server
Username: l
Password:
Username: l
Password was collected without echoing it.
Server:  ERROR|Invalid username or password
Authentication failed.
Please try again.

```

```

Username: alice
Password:
Username: alice
Password was collected without echoing it.
Server:  OTP REQUIRED

```

```

• $ python3 auth_server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 59122)
Received: AUTH|l|l
Received: AUTH|alice|Cyber123!
Current OTP: 659295
Password verified for: alice
Received: OTP|659295
OTP received from client: 659295
alice successfully authenticated
Received: MSG|hi there
alice says: hi there
Received: EXIT|
Server closed

```

Test 6

This should not be doable in the code as there is no separate input for OTP before entering the username and password, OTP is only available to be entered after the password phase. Cannot be demonstrated with the code as the code logic shows that it cannot be done regardless

Test 7

```
auth_message = "AUTH|" + username + "|" + password
```

This shows that you cannot have half the command in, password is adding into the sending to the server, if the password is empty then it will just return with error handling from server side, that being empty password and the client will be required to re-enter their user credential (can be seen from test 5's screenshot)

Part 14

1. Why does TOTP depend on synchronized time?

TOTP generates a one-time password using secret and the current time. The client and server must therefore use approximately the same time so they get the same, if their clock time are significantly different, they will generate different code

2. What happens when a code is submitted after its validity window?

The OTP becomes expired and the server rejects it as invalid. With a 30 second TOTP period, code generated in one time period normally cannot be used once the server has moved into a later time

3. Why might a small amount of clock tolerance sometimes be necessary?

Client and server clock may not be synchronized. Delays, processing time, or small differences in system clocks can cause a legit OTP to arrive just after the expected time window. Allowing a small neighboring time window can prevent legit users from being rejected

4. What security trade-off could occur if the server accepts too many neighboring time windows?

Accepting too many windows make OTP valid for too long period of time, increasing the opportunity for an attacker who gets an OTP to use it before it expires. The clock tolerance should be kept small to balance usability and security.

Part 15

1. Why should passwords not be stored in plaintext?

If the database is compromised, attackers would get users actual passwords, then log into the users accounts

2. What is the purpose of hashing a password before storage?

Hashing converts the password into a one-way value, the server can then compare the hash of a submitted password with the stored hash without storing the original password.

3. Why is plain SHA-256 alone insufficient for production password storage?

SHA-256 is designed to be fast, which make it vulnerable to fast brute-force and dictionary attacks, using a slow hashing algorithm like bcrypt with unique salt should be something production systems look into

4. What additional protection does TOTP provide beyond a password?

TOTP provides a second auth factor, an attacker who only get the password would still need the one time code to authenticate

5. How can TOTP reduce the usefulness of a captured one-time code after it expires?

TOTP code are valid for only short period of time like 30 seconds, once the time expires the old OTP would be rejected

6. What happens when the user and server clocks are significantly out of sync?

Generating OTP differently for the same account, legit user will be rejected by the server (answered in part 14)

7. What happens if an attacker obtains the user's TOTP secret?

The attacker can generate valid TOTP code themselves, if they know the user's password, they can bypass 2 factor authentication, the secret should be protected

8. Why should repeated failed login attempts be limited or monitored?

Rate limiting and monitoring helps prevent brute-force and rainbow table attacker, can also alert admins to suspicious login activity

9. Does `getpass()` protect the password while it travels across the network? Explain.

No, `getpass()` only prevents the password from being displayed on the user's screen while its being typed, the password is still sent through the raw TCP connection, an attacker who captures the network traffic could still see it, there for TLS would be needed to protect the password in transit.

10. What additional security would be required before deploying this authentication system in the real world?

1. The system would need several improvements, including:
2. TLS to encrypt network traffic
3. A password hashing algorithm such as Argon2, bcrypt, or scrypt
4. Secure storage and protection of TOTP secrets
5. Rate limiting and account lockout/monitoring
6. Secure session management
7. Input validation and protection against injection attacks
8. Logging and monitoring of authentication events
9. Secure key/secret management
10. Protection against replay and other authentication attacks
11. Proper error handling that does not reveal unnecessary information

Part 16

The authentication system I implemented uses password hashing and TOTP to provide two stages of authentication. The user database does not store the plaintext password. Instead, the password is processed with SHA-256 using the `hash_password()` function, and only the resulting hash is stored in the `user` dictionary. During login, the submitted password is hashed again and compared with the stored hash using `verify_password()`. This prevents the server from needing to store the original password, although SHA-256 alone is not suitable for production password storage because it is designed to be fast. An attacker with stolen hashes could perform large numbers of password guesses. A password-specific algorithm such as Argon2, bcrypt, or scrypt would provide stronger protection.

TOTP changes the authentication workflow by requiring the user to successfully verify their password before an OTP can be submitted. After the password is verified, the server retrieves the user's stored TOTP secret and checks the six-digit code generated for the current time period. Only after both factors are successfully verified does the server set `authenticated = True` and allow the user to send messages. This helps mitigate the risk of an attacker gaining access using only a stolen password. TOTP also limits the usefulness of a captured code because it expires after a short time.

However, the system still has several attack vectors. If an attacker obtains the user's TOTP secret, they can generate valid OTPs themselves, especially if they also know the password. Repeated authentication attempts could also allow password guessing or OTP guessing if attempts are not limited or monitored. The system should therefore use rate limiting, logging, and monitoring.

A major weakness of this implementation is that credentials are transmitted over a raw TCP connection. Although `getpass()` prevents the password from being displayed while it is entered, it does not encrypt the password or other authentication messages during transmission. Someone able to capture the traffic could potentially obtain the password and other sensitive information. For a real-world deployment, I would use TLS to protect the connection, replace SHA-256 with Argon2 or another password hashing algorithm, securely store TOTP secrets, add rate limiting and account monitoring, and implement stronger session management and input validation. These improvements would address the major limitations of the current lab implementation.

